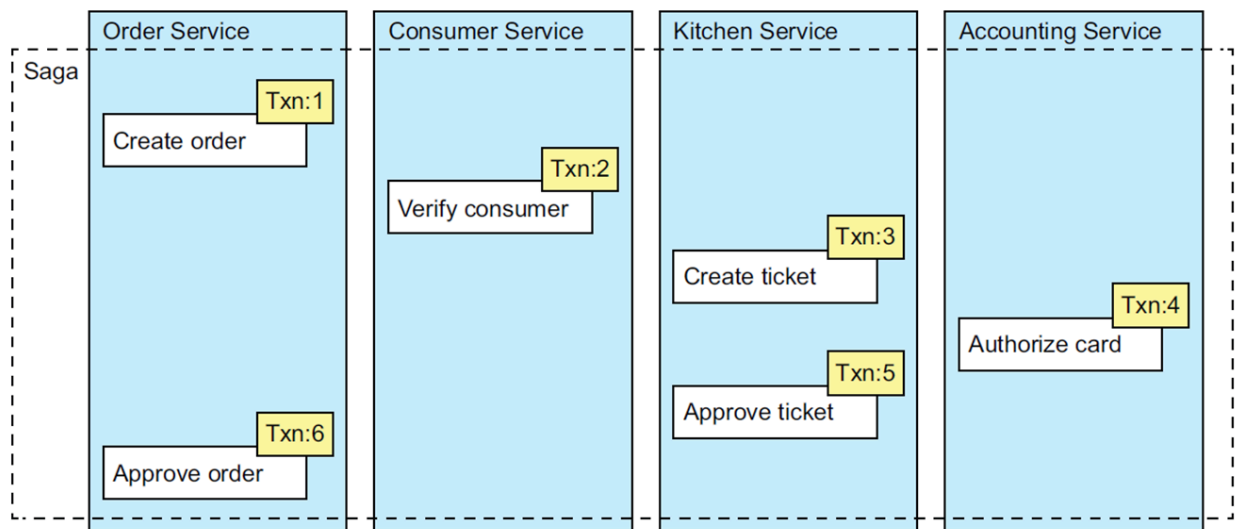


Lab 5 :Distributed Consistency via gRPC Orchestration

We will implement a simplified version of the following architecture:

1. **Order-Orchestrator**: The central controller.
2. **Order-Service**: Maintains the final order status.
3. **Kitchen-Service**: Handles ticket creation (**CREATE_PENDING**)."
4. **Accounting-Service**: Handles card authorization.



Step 1: The Protocol Definitions (.proto)

Define the services contracts. Note that every service has an **Execution** method and a **Compensation** method.

ORDER_SERVICE_PORT= 50051

KITCHEN_SERVICE_PORT= 50052

ACCOUNTING_SERVICE_PORT= 50053

1.3 order.proto

```
syntax = "proto3";
option java_multiple_files = true;
package com.gourmet.order;
service OrderService {
```

```

rpc UpdateStatus(UpdateStatusRequest) returns (UpdateStatusResponse); }

message UpdateStatusRequest {
    string orderId;
    string status; // e.g., "APPROVAL_PENDING", "APPROVED", "REJECTED"
}
message UpdateStatusResponse { bool acknowledged = 1; }

```

1.1 kitchen.proto

Protocol Buffers

```

syntax = "proto3";
option java_multiple_files = true;
package com.gourmet.kitchen;

```

```

service KitchenService {
    rpc CreateTicket(TicketRequest) returns (TicketResponse);
    rpc RejectTicket(RejectRequest) returns (RejectResponse); // Compensation }

```

```

message TicketRequest { string orderId; }
message TicketResponse { bool success = 1; }
message RejectRequest { string orderId; }
message RejectResponse { bool acknowledged = 1; }

```

1.2 accounting.proto

Protocol Buffers

```

syntax = "proto3";
option java_multiple_files = true;
package com.gourmet.accounting;

```

```

service AccountingService {
    rpc AuthorizeCard(AuthorizeRequest) returns (AuthorizeResponse); }
// authorized==true , if amount <100 else ==false

```

```

message AuthorizeRequest { string orderId; double amount; }
message AuthorizeResponse { bool authorized = 1; }

```

Step 2: Implementation Guidance

Task A: The "Happy Path" Logic

Implement the `OrderOrchestrator.java` based on the following steps:

1. Read:
 - `orderId` from the command line
 - `amount` from the command line
2. Create gRPC channels for all services
3. Create blocking stubs
4. Call `OrderService.UpdateStatus(orderId, "APPROVAL_PENDING")`.
5. Call `KitchenService.CreateTicket(orderId)`.
 - if `kitchenServiceResult == true`, continue to the next step
 - if `KitchenServiceResult == false`, stop the normal flow and throw an error
6. Call `AccountingService.AuthorizeCard(orderId, amount)`.
7. If all succeed: Call `OrderService.UpdateStatus(orderId, "APPROVED")`.

Task B: The "Undo" Logic (Compensation)

Modify the code to add the compensation logic:

If `AuthorizeCard` returns `authorized = false`, students must trigger the reverse flow:

1. Call `KitchenService.RejectTicket(orderId)`.
2. Call `OrderService.UpdateStatus(orderId, "REJECTED")`.